

7주차 같은 피처를 두 번 만들지 않으려면

7주차 · 피처 스토어

이 장의 수치는 모두 `practice/chapter7/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다. 다만 Feast 엔진이 낸 값(6절, 그리고 4-5절 끝의 대조 줄)은 이번에 다시 실행한 것이 아니라 **과거 실행이 남긴 산출물에서 옮긴 값**이다. 그 값의 출처는 `practice/chapter7/data/output/*.json` 이고, 나머지 수치는 이번에 실행해 만든 `practice/chapter7/results/*.log` 에서 옮겼다.

- ☐ 1주차는 데이터 엔지니어링과 MLOps가 만나는 자리가 피처(feature)이고, 바로 그 자리에서 학습-서빙 불일치가 생긴다고 예고했음
- ☐ 이 장이 그 예고를 회수함
- ☐ 6주차 배치가 확정한 일별 집계를 모델이 소비하는 계약으로 바꾸는 창구, 곧 피처 스토어(feature store)를 최소 구성으로 세움
- ☐ 이 장은 설명과 실습을 나누지 않음
- ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- ☐ 3~5절은 파이썬 표준 라이브러리만으로 돌아가므로 순서대로 실행하면 그 절의 수치를 직접 생성함

1. 오늘의 질문

- ☐ 훈련 코드와 서빙 코드가 각자 "전일 민원 건수"를 계산하면 무엇이 어긋나는가?
- ☐ 같은 피처를 왜 두 곳에 나누어 두고, 두 곳의 값이 같다는 것은 무엇으로 보이는가?
- ☐ 7월 2일 오전 9시의 훈련 샘플에 그날 하루치 집계를 붙이면 왜 안 되는가?
- ☐ 이 피처를 누가 책임지고, 그 값이 언제 알려진 것인지는 어디에 적어 두는가?

2. 이 장에서 실행할 것

- ☐ 피처 스토어는 본사 레시피로 운영되는 프랜차이즈 주방과 구조가 같음
- ☐ 속성을 하나씩 짚지어 볼 수 있으므로 아래 표로 대응시킴

프랜차이즈 주방	피처 스토어	다루는 절
지점마다 눈대중으로 만들지 않고 본사 레시피 하나로 조리함	피처를 한 번 정의하고 훈련과 서빙이 같은 정의를 읽음	3
창고에 쌓아 둔 재료 전량 / 오늘 조리대에 꺼내 놓은 분량	offline store(이력 전체) / online store(키별 최신값)	4
어제 낸 요리를 재현하면서 오늘 아침 들어온 재료를 쓰면 어제의 요리가 아님	미래 누출과 시점 조인	5
레시피 문서에 담당 부서와 재료 원산지를 적음	피처 정의서의 소유자·원천·타임스탬프 의미	6

표 7.1 주방의 네 가지 절차와 이 장의 네 개념

- ☐ 비유가 깨지는 지점을 한 줄로 밝힘
- 재료는 꺼내 쓰면 줄지만 피처 값은 여러 모델이 동시에 읽어도 줄지 않음
- 그래서 주방은 재고를 세면 관리가 되지만, 피처 스토어에서는 세는 대신 **두 곳의 값이 같은지를 따로 대조해야 함**
- ☐ 실행하기 전에 네 가지를 먼저 적음. 근거 칸을 비워 두지 않음

표 7.2 예측표 — 코드를 열기 **전에** 채운다

예측 항목	내가 적은 값	근거 한 줄	실측
훈련 샘플 4행으로 시점 조인을 하면 결과는 몇 행인가			
원천 9행을 온라인으로 적재하면 온라인 스토어는 몇 행이 되는가			
강남구의 두 샘플(7/2 09:00, 7/4 09:00)에 붙는 값은 각각 얼마인가			
오프라인 최신값과 온라인 조회값이 같을 것인가			

- ☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	7-2-count-skew.py	정제 단계별로 같은 날 건수를 세어 대조	3	없음
2	7-3-materialize-consistency.py	온라인 적재와 훈련-서빙 일관성 점검	4	없음
3	7-4-point-in-time-join.py	시점 조인과 미래 누출 대조	5	없음
4	7-1-feature-store-minimal.py	같은 흐름을 Feast 엔진으로 실행	6	feast·pandas·pyarrow 설치

- 1~3번은 파이썬만 있으면 돌아감. 표준 라이브러리만 쓰고 난수도 외부 접속도 없음
- 4번만 패키지 설치가 필요하며, 설치는 아래 한 줄로 함

```
python scripts/setup_practice.py 7 # 저장소 루트에서 실행 - feast·pandas·pyarrow를 설치함
```

- 4번은 설치가 끝난 뒤에만 실행한다
- 7-1-feature-store-minimal.py 는 feast 와 pyarrow 가 있어야 돌아간다. 없으면 아무것도 만들지 못하고 끝난다.
- 실행이 실패해도 지난 산출물은 남는다. 이 스크립트는 다시 만들 피쳐 저장소 폴더만 비우고, 산출물 JSON은 계산이 끝난 뒤 덮어쓴다.

3. 같은 숫자를 두 곳에서 세면 반드시 어긋난다

- "지역구별 전일 민원 건수"를 입력으로 쓰는 민원 급증 예측 모델을 생각함
- 피쳐 스토어가 없으면 이 수치는 두 곳에서 따로 산출됨
 - 훈련 파이프라인은 과거 데이터 위에서 집계함
 - 서빙 API는 요청이 올 때마다 운영 DB에 질의함
- 두 코드는 같은 것을 계산하려는 서로 다른 구현이고, 서로 다른 구현은 어긋남
- 어긋나는 이유는 실력 차이가 아니라 **정제 단계를 어디까지 적용했는가**의 차이임
- 6주차 정제는 물리 레코드에서 중복을 빼고, 다시 표준 코드로 매핑되지 않은 건을 뺐음
- 세 단계 중 어디서 세어도 "그날 민원 건수"라는 이름이 붙음
- 첫 번째 코드를 실행함. 이 절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter7
python code/7-2-count-skew.py
```

출처: practice/chapter7/results/7-2-count-skew.log

[2] 어디까지 정제하고 세느냐에 따라 같은 날의 '민원 건수'가 갈림

날짜	물리 레코드	중복제거 유효	표준코드 확정
2026-07-01	20	20	18
2026-07-02	18	17	16
2026-07-03	22	22	20

[3] 세 팀이 각자 '2026-07-02 민원 건수' 피처를 만들면

서빙 API	운영 DB를 그대로 셈	18
분석 부서	중복 1건만 제거	17
훈련 파이프라인	중복 + 미매핑 1건까지 제외	16
최댓값과 최솟값의 차 2건 - 확정 집계 대비 12.5%		
셋 다 버그가 아님: 같은 날의 지역구별 확정 집계는 마포구 5, 관악구 5, 강남구 6 합계 16		

[4] 모델이 확정 집계로 훈련되고 서빙은 물리 레코드를 세면

2026-07-01	훈련	18	서빙	20	차이	+2
2026-07-02	훈련	16	서빙	18	차이	+2
2026-07-03	훈련	20	서빙	22	차이	+2
3일 모두 같은 방향 - 평균 차이 +2.00건						
한쪽으로만 치우친 편의라서 평균 보정으로 사라지지 않음						

[5] 이 실행이 끝나는 방식

예외 없음, 종료 코드 0 - 세 값이 어긋난 채로 정상 종료함

□ 화면에 나온 값에서 읽을 것

- **같은 날짜에 세 개의 정답이 있음** — 7월 2일은 18·17·16으로 갈림. 어느 것도 계산 오류가 아니고, 세 값의 차이는 중복 1건과 미매핑 1건을 뺐는지 여부에서 나옴
- **차이가 12.5%임** — 확정 집계 16 대비 2건이 2건의 문제로 보이지 않는 이유가 여기 있음. 모델이 받는 입력이 8분의 1만큼 밀려 들어감
- **세 날 모두 차이가 +2로 같은 방향임** — 어긋남이 무작위로 흔들리는 것이 아니라 한쪽으로 치우친 편의임. 그래서 "평균을 내면 상쇄된다"는 대응이 성립하지 않음
- **종료 코드 0** — 세 값이 다른 채로 프로그램이 정상 종료함. 훈련 파이프라인도 서빙 API도 오류를 내지 않으므로 이 어긋남은 로그에 남지 않음

□ 모델이 16으로 훈련되고 18로 서빙되는 상태를 **훈련-서빙 스큐(training-serving skew)** 라 함

- 1주차가 학습-서빙 불일치라고 부른 것과 같은 현상이며, 이 장은 피처 계산의 관점에서 스큐라고 부름
- 두 코드가 오류 없이 계속 도는 탓에 성능 저하의 원인이 몇 주간 모델 쪽에서 탐색됨

□ 피처 스토어의 발상은 계산을 두 번 하지 않는 것임

- 한 번 정의하고, 한 곳에 저장하고, 훈련과 서빙이 같은 곳에서 조회함
- 구체적으로는 세 가지가 일원화됨

- **정의** — 어떤 원천에서 어떤 스키마로 계산되는지를 코드로 선언하고 버전을 붙임(6절)
- **저장** — 값은 피처 스토어에 있고, 훈련용 이력과 서빙용 최신값이 이중으로 유지됨(4절)
- **조회** — 훈련은 시점 조인으로, 서빙은 키 조회로 읽음. API는 달라도 같은 정의에서 나온 값임(5절)
- ☐ 피처 스토어가 16과 18 중 무엇이 옳은지를 판정하지는 않음
- ☐ 하나를 정의로 고르게 하고 모두가 그것만 쓰게 만들 뿐임
- ☐ 이 실습의 정의는 새 계산이 아니라 6주차까지 만들어 온 확정 집계임
- ☐ 곧 피처 스토어는 배치 파이프라인이라는 생산자와 모델이라는 소비자 사이의 창구다
- ☐ 반대 논거도 함께 확인함
- ☐ 모델이 하나뿐이고 배치 추론만 하며 피처 계산이 훈련 파이프라인 안에서 끝나면, 저장소를 하나 더 들이는 것은 관리 대상만 늘림
- 훈련과 서빙이 같은 코드에서 나오면 스코드가 성립하지 않기 때문임
- ☐ 이중 구조가 필요해지는 조건은 대체로 셋 중 하나임 — 온라인 서빙이 있어 소비 경로가 실제로 둘인 경우, 여러 모델이 같은 피처를 재사용하는 경우, 미래 누출 감사가 요구되는 경우

4. 저장소를 둘로 나누고 옮겨 담는다 — offline·online·materialize

- ☐ 훈련과 서빙은 같은 피처를 전혀 다르게 소비함
- ☐ 훈련은 "지난 석 달분, 모든 지역구, 모든 날짜"를 한 번에 읽음. 대량이고 과거 이력이 필요하며 몇 분 걸려도 됨
- ☐ 서빙은 "지금, 강남구, 최신값 하나"를 밀리초 안에 읽어야 함. 소량이고 최신값만 있으면 되며 지연을 허용하지 않음
- ☐ 하나의 저장소로 두 패턴을 모두 잘하기 어려워서 저장을 둘로 나눔
- ☐ **offline store**는 피처의 전체 이력을 쌓는 곳임
- ☐ 실습에서는 파일 하나가 그 역할을 하고, 운영에서는 데이터 웨어하우스가 흔히 쓰임
- ☐ **online store**는 키별 최신값만 두는 곳임
- ☐ 실습에서는 파이썬 사전, 7-1의 Feast 구성에서는 SQLite, 운영에서는 Redis 같은 키-값 저장소를 씀
- ☐ **materialize**는 오프라인의 값을 온라인으로 옮겨 담는 배치 작업임
- ☐ 이력 전체를 복사하는 것이 아니라 키마다 최신 1행만 남기는 작업임
- ☐ 두 번째 코드를 실행함. 일별 집계를 피처 행으로 펴고, 적재하고, 두 저장소를 대조함

python code/7-3-materialize-consistency.py

출처: practice/chapter7/results/7-3-materialize-consistency.log

[1] offline store – 피쳐 이력 전체 9행 (3일 × 지역구 3개)

lawd_cd	지역	알려진 시점	건수	미매핑율
11440	마포구	2026-07-02T00:00:00Z	5	0.050
11620	관악구	2026-07-02T00:00:00Z	5	0.050
11680	강남구	2026-07-02T00:00:00Z	8	0.050
...				
11440	마포구	2026-07-04T00:00:00Z	6	0.045
11620	관악구	2026-07-04T00:00:00Z	5	0.045
11680	강남구	2026-07-04T00:00:00Z	9	0.045

[2] materialize – 적재 창을 명시해서 호출

창 [2026-07-01T00:00:00Z, 2026-07-20T00:00:00Z]

창 안의 원천 9행 → online store 3행(키별 최신 1행)

11440	마포구	6	0.045	@2026-07-04T00:00:00Z
11620	관악구	5	0.045	@2026-07-04T00:00:00Z
11680	강남구	9	0.045	@2026-07-04T00:00:00Z

[3] 훈련-서빙 일관성 점검 – 오프라인 최신값 대 온라인 조회값

키	오프라인 최신	온라인 조회	일치
11440	6 / 0.045	6 / 0.045	예
11620	5 / 0.045	5 / 0.045	예
11680	9 / 0.045	9 / 0.045	예

checked_keys 3, all_consistent True

[4] 같은 적재를 증분 호출로 하면 – 시작점이 '기준 시각 - TTL'

기준 시각 2026-07-20T00:00:00Z(구성값), TTL 3일

창 [2026-07-17T00:00:00Z, 2026-07-20T00:00:00Z]

창 안의 원천 0행 → online store 0행

예외 없음, 오류 로그 없음 – 적재 작업은 성공으로 끝남

이 상태에서 온라인을 조회하면

키	오프라인 최신	온라인 조회	일치
11440	6 / 0.045	없음(None)	아니오
11620	5 / 0.045	없음(None)	아니오
11680	9 / 0.045	없음(None)	아니오

checked_keys 3, all_consistent False

□ 화면에 나온 값에서 읽을 것

○ **9행이 3행이 됨** — 예측표에 9를 적었다면 두 저장소의 성격 차이를 놓친 것임. 온라인에 9행이 남는 구성이라면 오히려 설정이 잘못된 것임

○ **온라인의 세 값이 7월 3일분임** — 마포 6·관악 5·강남 9는 마지막 날의 확정 집계이고, 온라인 스토어는 언제나 "그 키의 마지막으로 알려진 값"만 들고 있음

○ **적재 창을 바꾸자 0행이 됨** — [4]는 증분 적재가 쓰는 "기준 시각 - TTL" 규칙을 그대로 구현해 호출한 것임. 실습 피처의 타임스탬프가 그 창보다 과거라서 창 안에 원천이 없고, 옮길 것도 없음

○ **0행을 적재하고도 성공으로 끝남** — 예외도 오류 로그도 없음. 4주차의 유실, 5주차의 폐기, 6주차의 정제 drop에 이어 네 번째로 같은 교훈이 나옴. **비어 있는 결과는 스스로 알리지 않음**

○ **그래서 성공 로그가 아니라 적재된 건수를 봄** — 9행과 0행을 가르는 것은 종료 코드가 아니라 창 안의 원천 N행이라는 한 줄임

□ 같은 값이 두 곳에 있으므로, 두 곳이 정말 같은지가 곧 품질 항목이 됨

○ [3]의 표가 훈련-서빙 일관성 점검표이고, 세 키 전부에서 오프라인 최신값과 온라인 조회값이 같았음 (`checked_keys 3, all_consistent True`)

○ 같은 점검을 0행 적재 상태에 돌리면 세 키 모두 불일치로 잡힘

○ 곧 이 표는 침묵 실패를 값으로 바꾸는 장치임. 없으면 서빙이 `None` 을 받는 상태가 조용히 지나감

□ 만약 "값이 없으면 0으로" 같은 방어적 기본값이 서빙 코드에 있었다면 모든 지역구의 피처가 0인 채로 서빙됐을 것임

○ 결측은 기본값으로 덮지 말고 **피처 결측률**로 계수함(11주차)

□ 점검표가 자명해 보인다면 그것이 요점임

○ 자명함은 두 저장소를 같은 정의에서 적재했기 때문에 온 것이지 저절로 오지 않음

○ 3절의 각자 계산 구조에서는 이 표의 어느 행이든 어긋날 수 있음

□ 다만 저장된 값이 같다고 안심할 수는 없음

○ 서빙 코드가 값을 읽은 뒤 다르게 변형하면 모델이 받는 입력은 여전히 어긋남

○ 운영에서는 서빙 시점에 모델로 들어간 값을 로그로 남겨 훈련 분포와 정기 비교함(11주차)

□ 적재 주기를 정하는 순서는 거꾸로임

○ 먼저 모델이 견딜 수 있는 피처의 낡음을 정하고, 적재 주기를 그 안쪽으로 잡음

○ 낡음의 정도를 **피처 신선도**로 재는데, 온라인 스토어에 있는 값의 타임스탬프와 현재 시각의 차이임

○ 하루 한 번 적재하면 이 값의 최댓값은 약 24시간이 됨

□ 24시간이 괜찮은지는 두 가지가 정함

○ 피처가 가리키는 현상이 얼마나 빨리 변하는가

○ 그 값으로 내린 결정을 되돌리는 비용이 얼마인가

○ "전일 민원 건수"로 다음 날 인력 배치를 정한다면 하루 지연은 정의상 문제가 아님

○ 재난 신고 밀집도로 출동 순서를 정한다면 24시간 전 값은 쓸 수 없고, 잘못 정한 순서를 되돌릴 방법도 없음

□ 같은 흐름을 Feast 엔진으로 실행한 결과도 같은 값을 남겼음

○ `all_consistent` 는 true, `checked_keys` 는 3이고 키별 값은 마포 6·관악 5·강남 9, 미매핑율은 세 곳 모두 0.045임

```
# 출처: practice/chapter7/data/output/ch7_consistency_report.json (과거 Feast 실행 산출물)
{
  "all_consistent": true,
  "checked_keys": 3,
  "rows": [
    {
      "consistent": true,
      "lawd_cd": "11440",
      "offline_latest": {
        "complaint_count": 6,
        "day_unmapped_rate": 0.045
      },
      "online": {
        "complaint_count": 6,
        "day_unmapped_rate": 0.045
      }
    },
    ... (11620, 11680 두 행 생략)
  ]
}
```

- 두 값이 같다고 엔진을 다시 돌린 것은 아니다
- 위 JSON은 과거에 Feast로 실행해 남긴 산출물이고, 이번에 실행한 것은 같은 규칙을 파이썬으로 구현한 7-3 이다.
- 값이 일치한다는 사실이 말해 주는 것은 적재 규칙을 같게 구현했다는 것뿐이다. Feast의 동작을 이 장에서 다시 검증한 것은 아니다.

5. 그 시점에 알 수 있었던 값만 붙인다 — 시점 조인

- 이제 훈련 데이터를 만들
- 훈련 샘플 하나는 "7월 2일 오전 9시의 강남구" 같은 키와 시점의 쌍이고, 여기에 "전일 민원 건수" 피처를 붙여야 함
- 이때 7월 2일 하루 전체의 집계를 붙이면, 그 값은 7월 2일 오전 9시에 아직 존재하지 않던 값임. 그날이 끝나야 확정되기 때문임
- 미래의 정보가 과거 시점의 샘플에 스며든 것을 **미래 누출(future leakage)** 이라 함
- 누출된 모델은 훈련과 평가에서 비현실적으로 좋게 나오다가 서빙에서 성능이 급격히 떨어짐
- 성능이 좋아 보여서 더 위험한 결함임
- **시점 조인(point-in-time join)** 이 이것을 조화 규칙으로 막음
- 각 샘플의 기준 시점마다 **그 시점 이전에 알려진 가장 최근 값만** 붙임
- 그래서 피처의 타임스탬프가 결정적으로 중요해짐 — 그 타임스탬프는 "값이 가리키는 기간"이 아니라

"값이 알려진 시점"이어야 함

□ 실습의 설계 결정이 정확히 이것임

○ 7월 1일의 일별 집계는 그날이 끝나야 확정되므로 타임스탬프를 7/1이 아니라 **7/2 00:00**으로 부여했음

○ 2주차의 이벤트 시간과 처리 시간, 5주차의 워터마크, 6주차의 logical date에 이어 네 번째로 확인하는 시간 구분임. 이번에는 발생한 시각도 처리한 시각도 아닌 "알 수 있게 된 시각"임

□ 세 번째 코드를 실행함. 기준 시점이 다른 훈련 샘플 4개에 두 가지 조인 규칙을 각각 적용함

python code/7-4-point-in-time-join.py

출처: practice/chapter7/results/7-4-point-in-time-join.log

[2] 시점 조인 - 훈련 샘플 4행

11680 as_of 2026-07-02T09:00:00Z → 건수 8, 미매핑율 0.050

쓴 피처: 2026-07-01분, 알려진 시점 2026-07-02T00:00:00Z (간격 0일 9시간)

11440 as_of 2026-07-03T09:00:00Z → 건수 5, 미매핑율 0.059

쓴 피처: 2026-07-02분, 알려진 시점 2026-07-03T00:00:00Z (간격 0일 9시간)

11680 as_of 2026-07-04T09:00:00Z → 건수 9, 미매핑율 0.045

쓴 피처: 2026-07-03분, 알려진 시점 2026-07-04T00:00:00Z (간격 0일 9시간)

입력 4행 → 결과 3행 (탈락 1행)

탈락: 11620 as_of 2026-07-01T09:00:00Z - 확정된 값 없음

탈락 행은 빈 값으로 남지 않고 결과에서 사라짐 - 오류도 경고도 없음

[3] 같은 입력에 최신값 조인을 하면 - 미래 누출

lawd_cd	as_of	시점 조인	최신값 조인
11620	2026-07-01T09:00:00Z	탈락	5
11680	2026-07-02T09:00:00Z	8	9
11440	2026-07-03T09:00:00Z	5	6
11680	2026-07-04T09:00:00Z	9	9

4행 모두 값이 붙어 훈련 셋은 줄지 않지만, 어긋난 행 3개

[4] TTL 하한 - 알려진 값이어도 오래되면 붙이지 않음

11680 as_of 2026-07-10T09:00:00Z

as_of 이전 최신 = 2026-07-04T00:00:00Z (건수 9)

결과: 탈락 - 가장 최근 값이 6일 9시간 전 - TTL 3일 초과

□ 둘째 줄을 규칙 그대로 손으로 따라가 봄

○ 강남구(11680)의 피처 값은 7/2 00:00, 7/3 00:00, 7/4 00:00에 각각 존재함

○ 기준 시점 7/2 09:00 이전인 것은 7/2 00:00 하나뿐이므로 7/3과 7/4의 값은 미래라서 후보에서 빠짐

○ 남은 후보의 값 8이 붙고, 기준 시점과의 간격 9시간은 TTL 3일 안이라 유효함

○ 출력의 8이 손 계산과 같음

□ 화면에 나온 값에서 읽을 것

○ **같은 키에 다른 값이 붙음** — 강남구 샘플 둘에 8과 9가 붙었음. 같은 키라도 기준 시점이 다르면 그때 알 수 있었던 값이 다르기 때문이고, 이것이 시점 조인의 정의임

○ **두 값이 같아지면 그것이 누출임** — [3]의 최신값 조인은 7/2 09:00 샘플에도 9를 붙임. 예측표에 두 값을 같게 적었다면 지금 하려는 것은 시점 조인이 아니라 최신값 조인임

○ **누출은 행 수로 드러나지 않음** — 최신값 조인은 4행을 그대로 4행으로 내주고 어긋난 행만 3개임. 훈련 셋이 커 보이는 쪽이 잘못된 쪽임

○ **행이 통째로 사라짐** — 7/1 09:00의 관악구 샘플은 피처가 빈 값으로 남는 것이 아니라 결과에서 없어짐. 입력 4행에 결과 3행이고 오류는 나지 않음

□ 조용한 샘플 손실을 어떻게 다루지는 이 한 줄에서 갈림

○ 예측표에 4를 적었다면 유효 피처가 없는 샘플이 빈 값으로 남는다고 본 것임

○ 손실이 무작위가 아니라는 점이 더 문제임 — 이력이 짧은 신규 키의 샘플이 체계적으로 빠지므로 **편향된 손실임**

○ 대응은 6주차 정제의 총계 보존과 같은 구조임. 조인 전후의 행 수를 반드시 대조하고, 차이가 나면 어느 샘플이 왜 빠졌는지 확인함

□ 시점 조인에는 하한도 있음

○ [4]가 그것으로, TTL 3일을 넘긴 값은 그 시점에 알려져 있었더라도 붙지 않음

○ 석 달 전의 "전일 민원 건수"는 알려진 값이긴 해도 "전일"이라는 피처의 의미를 잃기 때문임

○ 5주차의 워터마크가 "얼마나 늦은 데이터까지 받을 것인가"라면 TTL은 "얼마나 오래된 피처까지 유효로 볼 것인가"임. 둘 다 선언 하나로 시간의 경계를 정하고 경계 밖을 제외함

□ 그래서 TTL은 디스크나 성능이 아니라 **배치 실패를 며칠까지 견딜 것인가**에서 역산함

○ 실습의 요구사항은 "배치가 하루 실패한 적은 있고 이를 연속 실패한 적은 없다"였음

○ TTL을 1일로 잡으면 하루 실패한 다음 날부터 유효한 값이 없어 훈련 샘플이 통째로 빠짐

○ 3일은 이를 연속 실패까지 견디고 하루의 여유를 남긴 값임

○ 길게 잡으면 안전하지만 오래된 값이 "전일 건수"라는 이름을 달고 붙는 신선도 손실을 받음

□ 기준 시점을 어디로 잡을지도 결과를 바꿈

○ 실습은 각 날짜의 09:00을 썼는데, 임의의 값이 아니라 "오전 9시에 그날의 인력 배치를 정한다"는 업무 시나리오에서 나온 구성값임

○ 결정은 09:00에 내리면서 훈련 샘플의 기준 시점을 23:59로 두면 그날 낮에 확정된 값이 붙어 설정값 하나로 미래 누출이 들어옴

○ 반대로 자정 기준으로 훈련하고 09:00에 서빙하면 누출은 없지만 서빙 쪽이 더 신선한 값을 받아 스쿼의 시간 버전이 됨

○ 규칙은 하나다 — **훈련 샘플의 기준 시점은 운영에서 실제로 예측이 호출되는 시각과 같아야 한다**

□ 같은 조인을 Feast 엔진으로 실행한 결과도 같은 값을 남겼음

○ 입력 4행에 조인 결과 3행, 탈락 1행이고 붙은 값은 8·5·9임

```
# 출처: practice/chapter7/data/output/ch7_feature_report.json (과거 Feast 실행 산출물)
"entity_rows_requested": 4,
"historical_dropped_rows": [
  {
    "as_of": "2026-07-01 09:00:00+00:00",
    "lawd_cd": "11620"
  }
],
"historical_join": [
  {
    "as_of": "2026-07-02T09:00:00+00:00",
    "complaint_count": 8,
    "day_unmapped_rate": 0.05,
    "lawd_cd": "11680"
  },
  ... (11440 7/3, 11680 7/4 두 행 생략 - 각각 5와 9)
```

6. 피처 정의서 — 누가 책임지고 언제 알려진 값인가

- 피처가 한 곳에 정의되고 여러 모델이 재사용하는 자산이 되는 순간, 기술 문제와 함께 책임 문제가 옴
- 원천 부서가 집계 방식을 바꿨는데 소비하는 모델 셋이 그것을 모르면 사고가 남
- 그래서 피처 정의서에는 계산 명세만이 아니라 소유와 책임이 들어감
- **소유자** — 이 피처의 정의와 품질에 책임지는 부서임
- 원천이 바뀌면 소비자에게 알릴 의무가 여기에 붙음
- **원천과 갱신 주기** — 어느 시스템의 어떤 산출물에서 얼마나 자주 만들어지는가
- **개인정보 등급** — 피처 단위로 표기함
- 지역 단위 집계는 비식별이지만 가구·개인 단위 피처는 접근 통제와 이용 목적 제한이 붙음
- 등급이 피처 단위로 붙는다는 점이 중요함. "피처 스토어에 접근 가능"이라는 시스템 단위 권한은 개인정보 피처 앞에서 무의미함
- **타임스탬프 의미** — 5절의 "알려진 시점" 규약을 문서에 적음
- 미래 누출 감사에 답할 때 근거가 되는 문장임
- 네 번째 코드를 실행함. 이 절의 값은 과거 실행이 남긴 산출물에서 옮긴 것임

```
python scripts/setup_practice.py 7 # 저장소 루트에서, 먼저 설치
cd practice/chapter7 && python code/7-1-feature-store-minimal.py
```

```
# 출처: practice/chapter7/data/output/ch7_feature_definitions.json (과거 Feast 실행 산출물)

"entity": {
  "description": "법정동 시군구 코드(행정표준코드)",
  "join_key": "lawd_cd",
  "name": "district"
},
"feature_service": {
  "name": "complaint_model_v1",
  "views": [
    "complaint_daily"
  ]
},
"feature_view": {
  "features": [
    {
      "description": "전일 확정 민원 건수(7장 일별 집계)",
      "dtype": "Int64",
      "name": "complaint_count"
    },
    ... (day_unmapped_rate 생략)
  ],
  "governance": {
    "owner": "민원데이터팀(가상 - 시뮬레이션 표기)",
    "privacy": "개인정보 없음(지역 단위 집계)",
    "source": "7장 일별 배치 확정 산출물",
    "update_cycle": "일 1회(배치 마감 후)"
  },
  "name": "complaint_daily",
  "timestamp_semantics": "event_timestamp = 값이 확정·공개된 시점(집계 대상일 다음 날 00:00 UTC)",
  "ttl_days": 3
},
"project": "public_complaints"
```

□ 화면에 나온 값에서 읽을 것

○ 정의서가 사람이 쓴 문서가 아님 — 이 JSON은 등록된 정의를 registry에서 그대로 뽑은 것임. 코드와 문서가 어긋날 수 없는 이유가 여기 있음

○ 조인 키가 지역명이 아니라 코드임 — 6주차의 "이름이 아니라 코드로 붙인다"는 표준 코드 원칙이 entity 설계에서 그대로 반복됨

○ ttl_days 가 정의서에 있음 — 5절에서 배치 실패 견딤으로 역산한 3일이 문서에도 값으로 남음. 조인

결과가 이상할 때 먼저 확인할 숫자임

○ 소유자가 "가상"으로 표기됨 — 실습의 조직명은 시뮬레이션 표기이며, 실제 운영에서는 연락 가능한 부서가 들어가야 이 칸이 뜻을 가짐

○ source 와 설명에 적힌 "7장"은 참고 원고의 장 번호이고, 강의 주차로는 6주차의 일별 배치 산출물을 가리킴

□ entity는 형식상 조인 키의 선언이지만, 실제로 고르는 것은 피처가 서술하는 대상의 단위임

○ 무엇이 옳은지는 서빙 시점에 무엇을 키로 조회하는가가 정함

– 상황실이 "강남구의 현재 상태"를 물으면 키는 지역구여야 함

– 접수 창구가 "이 민원 건의 예상 처리 시간"을 물으면 키는 민원 건 ID여야 함

○ 단위를 잘게 잡으면 피처는 정밀해지지만 온라인 스토어의 키 개수가 늘어 적재 비용이 커짐

○ 키마다 이력이 짧아져 5절의 행 탈락도 늘어남

□ 정의는 불변이 아님

○ 정제 규칙이 바뀌면 같은 이름의 피처가 다른 값을 갖게 됨

○ 기존 정의를 조용히 덮어쓰면 그 피처로 훈련된 모델들의 훈련-서빙 일치가 소급으로 무효가 됨. 스큐를 막으려던 구조가 스큐의 원인이 되는 역설임

○ 그래서 정의 변경은 새 feature service 버전으로 내고, 기존 버전은 소비 모델이 이전을 마칠 때까지 유지함

○ 실습 service 이름 complaint_model_v1 의 v1은 장식이 아니라 이 규율의 자리 표시임

□ 이 정의서는 7주차의 종착점이 아니라 다음 주차의 입력임

○ 9주차는 이 피처로 만든 학습 데이터의 지문 해시를 실험 기록에 남김

○ 13주차 검수표는 통합 시스템이 실제로 처리한 데이터의 지문을 그 값과 대조함

○ 두 값이 일치해야 "9주차에 학습에 쓴 데이터"와 "운영이 처리한 데이터"가 같다고 말할 수 있다

○ 정의서가 무엇을 계산했는지를 문장으로 남긴다면, 지문 해시는 그 계산의 결과가 바뀌지 않았음을 값 하나로 남김

7. 핵심 요약

□ 1. 피처를 두 곳에서 따로 계산하면 반드시 어긋나고, 그 어긋남은 오류로 드러나지 않는다. (3절)

○ 7월 2일 민원 건수는 어디까지 정제했느냐에 따라 18:17:16으로 같았고 셋 다 버그가 아니었음

○ 세 날 모두 차이가 +2로 같은 방향이라 평균 보정으로 사라지지 않고, 프로그램은 종료 코드 0으로 끝났음

○ 피처 스토어는 무엇이 옳은지 판정하지 않고 하나를 정의로 고르게 한 뒤 훈련과 서빙이 그것만 읽게 만듦

□ 2. 소비 패턴이 달라서 저장소를 둘로 나누고, 적재는 건수로 확인한다. (4절)

○ 이력 9행이 offline store에 있고 materialize가 옮긴 최신 3행이 online store에 남았음

○ 적재 창을 증분 기본값으로 바꾸자 0행이 적재됐는데 예외도 오류 로그도 없었음

○ 그래서 적재는 성공 로그가 아니라 창 안의 원천 N행 으로 확인함

☐ 3. 두 저장소의 값이 같다는 것은 대조해서 보이는 것이다. (4절)

- 세 지역구에서 오프라인 최신값과 온라인 조회값을 비교해 `checked_keys` 3, `all_consistent` True를 얻었음
- 같은 점검을 0행 적재 상태에 돌리면 세 키 모두 불일치로 잡히므로, 이 표가 침묵 실패를 값으로 바꾸는 장치임
- 자명해 보이는 일치는 같은 정의에서 적재했기 때문에 온 것이지 저절로 오지 않음

☐ 4. 훈련 데이터에는 그 시점에 알 수 있었던 값만 붙인다. (5절)

- 타임스탬프를 "값이 알려진 시점"으로 두고 기준 시점 이전의 최신 값만 조인하니 강남구 두 샘플에 8과 9가 붙었음
- 최신값 조인으로 바꾸면 4행이 그대로 남고 어긋난 행만 3개가 되므로, 훈련 셋이 커 보이는 쪽이 누출된 쪽임
- 조인할 값이 없는 샘플은 행째 사라지고 TTL 3일을 넘긴 값도 붙지 않으므로, 조인 전후 행 수 대조는 선택이 아니다

☐ 5. 정의서는 피처가 자산이 되는 순간 필요해진다. (6절)

- 소유자·원천·갱신 주기·개인정보 등급·타임스탬프 의미를 registry에서 뽑아 문서로 남겼음
- 정의가 바뀌면 덮어쓰지 않고 새 버전으로 내며, `complaint_model_v1` 의 v1이 그 자리 표시임
- 이 문서와 9주차의 지문 해시가 함께 있어야 "무엇으로 학습했는가"에 답할 수 있다

8. 과제

☐ 3~5절의 코드를 순서대로 실행해 만든 로그 세 개를 LMS에 제출한다

1. `practice/chapter7/results/7-2-count-skew.log`
2. `practice/chapter7/results/7-3-materialize-consistency.log`
3. `practice/chapter7/results/7-4-point-in-time-join.log`

☐ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 표 7.2에 적은 예측 네 개와 실측을 나란히 적고, 어긋난 항목마다 무엇을 잘못 가정했는지 밝힌다.
2. 본인 로그 7-3 의 [3]에서 `checked_keys` 와 `all_consistent` , 그리고 키 하나의 오프라인·온라인 값을 옮겨 적는다.
3. 6주차의 일별 배치가 **나흘 연속 실패**했다고 하자. 나흘째에 온라인 스토어를 조회하면 무엇이 반환되고, 같은 시점에 오프라인 훈련 셋을 만들면 어떤 문제가 생기는가. 본인 로그의 [4] 블록과 TTL 3일을 근거로 답한다.
4. 이 사고를 배치 실패 직후에 잡으려면 어느 지점에 무엇을 두어야 하는가. 감지 지점 두 개를 제시하고 각각이 무엇을 신호로 삼는지 쓴다.

☐ 값을 지어내지 않는다. 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

- 실습을 더 해 보려면 `practice/chapter7/` 에서 다음을 시도함
- `7-4-point-in-time-join.py` 의 `TTL` 을 1일로 바꿔, 조인 결과가 몇 행으로 줄고 어느 샘플이 빠지는지 확인함
- `7-4-point-in-time-join.py` 의 `SAMPLES` 에서 기준 시점을 09:00에서 23:59로 바꿔, 붙는 값이 어떻게 달라지는지 확인함
- `7-3-materialize-consistency.py` 의 `RUN_AS_OF` 를 여러 값으로 바꿔, 증분 적재가 몇 행을 옮기는 시점부터 온라인이 비지 않는지 찾음
- 패키지를 설치했다면 `7-1-feature-store-minimal.py` 를 실행해 Feast 엔진의 결과가 위 세 로그와 같은지 직접 대조함
- 뒤 주차에서 이어지는 내용
- 피처 결측률과 서빙 입력 분포 비교는 11주차, 피처 접근 등급과 카탈로그·감사 기록은 12주차에서 다룸
- 학습 데이터의 지문 해시는 9주차에서 만들고 13주차 검수표에서 대조함
- 원문을 직접 보고 싶다면 다음 두 편이 출발점임
- Feast Documentation — entity·feature view·시점 조회의 공식 설명. <https://docs.feast.dev/>
- Zinkevich, 「Rules of Machine Learning」 — 훈련·서빙 스큐와 그 대응을 규칙 형태로 정리한 문서. <https://developers.google.com/machine-learning/guides/rules-of-ml>